Mitigating Inter-Process Communication Overhead in seL4-Based Systems: An Asynchronous, Multiclient File Server

A REPORT SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Luke Rule

Supervised by James Garside

Department of Computer Science

Abstract

TODO: my abstract

Contents

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Aims and Objectives 2

1.3 Evaluation Strategy 3

1.4 Key Findings 4

1.5 Dissertation Structure 4

2 Background **5**

2.1 Operating System Kernels 5

2.1.1 Definition and Characteristics 5

2.1.2 Microkernels Versus Monolithic Kernels 6

2.2 seL4 Microkernel 7

2.2.1 Overview and Characteristics 7

2.2.2 Design 8

2.2.3 Use Cases and Applications 9

2.2.4 Microkit Framework for seL4 9

2.3 Client-Server Communication 10

2.3.1 Synchronous Model 10

2.3.2 Asynchronous Model 11

2.4 File Servers in Operating Systems 12

2.4.1 Role and Functionality 12

2.4.2 Abstractions 12

2.4.3 Storage and Organisation 13

File Layout 13

Allocation Strategies 13

Metadata Organisation 15
Directory Structure 17
2.4.4 Client Management 17
2.5 Related Work 18

3 Design **19**
3.1 System Requirements and Constraints 19
3.2 IPC Design and File Server Interface 20
3.2.1 Synchronous Model *(Baseline)* 20
3.2.2 Asynchronous Model *(Proposed IPC Mitigation Strategy)* 20
3.3 File Server 20
3.3.1 Architectural Overview 20
3.3.2 Component Design and Interactions 20
3.3.3 Multiclient Support 20
3.3.4 File and Directory Operations 20

4 Implementation **21**
4.1 Development Environment and Tools 21
4.2 Microkit Framework Setup 21
4.3 Interface 23
4.3.1 Synchronous Server Model 23
Client-Server Communication Protocol 23
IPC Mechanism 23
API 23
4.3.2 Asynchronous Server Model 23
Client-Server Communication Protocol 23
IPC Mechanism 23
API 23
4.4 File Server 23
4.4.1 Data Structures and Access Methods 23
inodes 23

Blocks 24
Directories 24
File Descriptors 24
4.4.2 Operations 24
Entry Creation 24
File Access 24
Metadata Access and Modification 24
Entry Deletion 24
4.5 Error Handling 24

5 Evaluation **25**
5.1 Validation 25
5.1.1 Unit Tests 25
5.1.2 Integration Tests 26
Single Client 26
Multiple Clients 26
5.2 Performance Evaluation 27
5.2.1 Goals 27
5.2.2 Metrics 27
5.2.3 Experimental Setup 27
5.2.4 Benchmark Design 28
5.2.5 Results and Analysis 30
Single-Client Per-Operation Benchmark 30
Multiclient Realistic Workload 32

6 Conclusion **34**
6.1 Contributions 34
6.1.1 System Design and Implementation 34
6.1.2 IPC Overhead Mitigation Insights 35
6.2 Drawbacks and Limitations 36
6.2.1 File System Limitations 36

6.2.2 Benchmarking Constraints 36
6.2.3 Lack of Persistent Storage 36
6.2.4 Single-Threaded Execution 37
6.2.5 Dependency on Workload Characteristics 37
6.2.6 Additional Client Burden 37
6.3 Future Work 38

Bibliography **39**

A Appendix Title **40**
A.1 Example First Section 40
A.2 Output 40

Word Count: 15,000

List of Figures

1.1 Comparison of tightly coupled and loosely coupled system structures. 1

2.1 View of kernel interaction within an operating system. 5

2.2 Comparison of microkernel and monolithic kernel architectures. 6

2.3 Comparison of ACLs and Capability Based Access Control Models. 8

2.4 Build-time and Run-time Structures for a Microkit-based System. 9

2.5 Communication Channels Between PDs. 10

2.6 Comparison of Synchronous and Asynchronous Communication Models. 10

2.7 Multiclient File Server Structure. 12

2.8 Contiguous Allocation Example. 14

2.9 Linked Allocation Example. 14

2.10 Indexed Allocation Example. 15

2.11 File Allocation Table Example. 16

2.12 Indirect Block Indexing Example. 16

2.13 Directory Structure Example. 17

5.1 Average Time per Operation vs Batch Size (Read and Write) 30

5.2 Total Execution Time by Number of Clients (Synchronous vs Asynchronous) 32

5.3 Execution Times per Client (Fairness Analysis) 32

Declaration

No portion of the work referred to in this report has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on presentation of Theses

Acknowledgements

I would like to thank...

Abbreviations

PD Protection Domain

Introduction

1.1 Context and Motivation

Modern software systems are larger, more complex and more interconnected than ever before [**?**], placing significant demands upon the operating systems that underpin them. Contemporary mainstream operating systems, such as Windows, Linux and macOS [1] are largely based upon monolithic kernel designs[**?**], where the majority of their functionality resides in a single privileged address space. This key characteristic enables high performance[**?**], yet it comes at the cost of scalability, security and robustness[**?**], as it requires a large Trusted Computing Base (TCB) where a single fault can compromise the entire system[**?**].

Simultaneously, modern software and hardware architectures increasingly employ modularity, isolation and concurrency[**?**]. Distributed systems, cloud-native microservices and data centre-scale applications all decompose functionality into loosely-coupled components[**?**], as illustrated in Figure 1.1, and most current hardware performance scaling is driven by multi-core and many-core parallelism[**?**]. These trends place pressure on operating system design to provide stronger security guarantees and better support for concurrent, component-based execution.

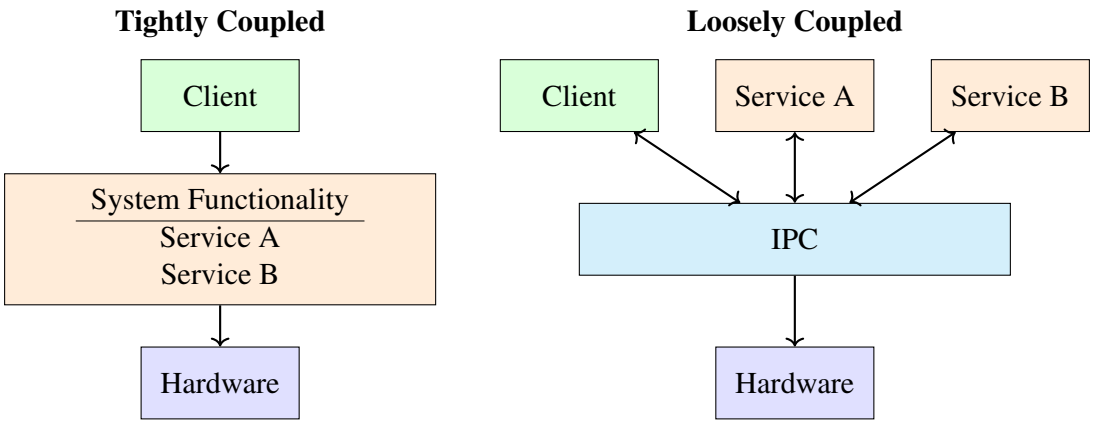

Figure 1.1: Comparison of tightly coupled and loosely coupled system structures.

[1] While macOS is often described as a hybrid kernel[**?**], it retains many monolithic characteristics to such an extent that it loses many of the benefits typically associated with pure microkernel design[**?**].

Microkernel design naturally aligns with these requirements[**?**]. By minimising kernel functionality and moving most non-essential services, such as device drivers and file systems, into user space, microkernels reduce the TCB significantly and enforce separation between components. However, this modularity requires costly Inter-Process Communication (IPC) across protected domains.

The L4 family of microkernels[**?**] addressed this bottleneck by substantially reducing IPC costs through aggressive kernel minimisation and IPC fastpath design[**?**]; yet, despite these advances, IPC overhead remains a fundamental constraint in microkernel-based systems and a barrier to their wider adoption[**?**]. However, by combining these performance improvements with strong security guarantees through formal verification[**?**], the seL4 microkernel represents a significant step towards making microkernel-based designs viable for real-world deployment, particularly in security and safety-critical domains[**?**]. Still, performance constraints remain a limiting factor, especially for high-frequency IPC workloads.

One of the most frequently accessed operating system services is the file server[**?**], which acts as the primary interface for persistent storage and data exchange. File system operations are both latency-sensitive and IPC-intensive[**?**], making them critical to overall system performance in microkernel environments. In synchronous architectures, even simple file operations may involve multiple IPC invocations between the client and the file server, amplifying the impact of communication overhead.

This report presents the design and implementation of an asynchronous, multiclient file server for the seL4 microkernel, validates its correctness and usability, and evaluates its effectiveness in mitigating IPC overhead. The proposed architecture decouples request submission from response handling, reducing blocking behaviour, increasing concurrency and enabling batches of operations to be sent and serviced at once. These properties have the potential to improve throughput and resource utilisation under both single and multiclient workloads. In addition, beyond serving as an evaluation platform, the implementation constitutes a practical, reusable system component, demonstrating how critical services can be structured within a microkernel environment using the seL4 Microkit's component-based design model.

1.2 Aims and Objectives

Building upon the L4 family's reduction of IPC overhead, and motivated by the increasing importance of concurrency in contemporary systems, this work aims to further mitigate IPC costs by reducing the number of interactions required between components. In doing so, it also seeks to demonstrate the capabilities of the seL4 Microkit framework through the design and implementation of a flexible, user-space file server.

This project has the following objectives:

- **Design and implement**:
  - A viable file system core capable of supporting a wide range of file and directory operations.
  - An efficient storage model based on inode and block abstractions, supporting effective space utilisation and flexible file and directory representation.
  - A scalable, asynchronous interface enabling concurrent request submissions and completion handling from multiple clients, alongside a synchronous interface to facilitate comparison between communication models.
  - A lightweight client-side library providing a simple and usable abstraction over the file server interface.
  - A benchmarking framework to evaluate asynchronous file server performance against a synchronous baseline under realistic workloads.
- **Validate**: the correctness of the file server through comprehensive multi-level testing.
- **Evaluate**: the effectiveness of the asynchronous file server architecture in reducing IPC overhead and improving performance, in terms of latency, throughput, and scalability.

1.3 Evaluation Strategy

To evaluate the proposed implementation, this work considered both the correctness of file system operations and the performance characteristics of asynchronous queue-based submission compared to synchronous, blocking calls.

Correctness was validated at multiple stages of granularity – unit, integration and end-to-end testing – to ensure file operations correctly persisted and returned data across a range of scenarios, including edge and error cases.

For performance evaluation, a set of benchmarks was designed to compare the synchronous and asynchronous interfaces under varying workloads and measure key metrics, including latency, throughput, and IPC frequency. Experiments were conducted across different levels of request batching to assess the trade-off between asynchronous overhead in low-batching scenarios and performance gains achieved through reduced IPC at higher batching levels.

The benchmark workloads were designed to approximate realistic file system usage patterns; however, as the system is novel, custom benchmarks were required. While these may not have captured all characteristics of production environments, they were sufficient to provide indicative and comparative insights into the performance impact of asynchronous design.

1.4 Key Findings

The evaluation demonstrates that asynchronous communication introduces negligible overhead compared to synchronous execution when no batching is applied, with per-operation latencies remaining effectively equivalent. Crucially, as batching is introduced, significant performance improvements are observed. For single-client workloads, increasing batch sizes reduces per-operation latency by up to 60% for reads and 50% for writes, demonstrating that reducing IPC frequency has a substantial impact on performance.

These improvements extend to more realistic workloads. In a multiclient benchmark simulating typical file system operations with batching opportunities, the asynchronous model achieves an average latency reduction of approximately 35–40% compared to the synchronous baseline. Notably, these experiments were conducted on a single core, as current seL4 deployments do not fully utilise multicore execution; consequently, greater throughput would be expected from asynchronous designs in a multicore setting due to increased concurrency, allowing requests to be issued and responses processed in parallel.

However, all evaluations were performed using an in-memory storage model rather than disk-backed persistence. While this isolates IPC and scheduling overhead, real-world performance gains would be less pronounced when incorporating variable disk I/O latency. Nevertheless, the results demonstrate that batching and non-blocking execution can significantly reduce IPC overhead, particularly in high-frequency workloads.

1.5 Dissertation Structure

The report begins by presenting the background knowledge required to understand the design of the file server and its architectural context. It then introduces the system design, including the overall architecture, interface models, and key design decisions.

Following this, the implementation of the file server and its two interfaces are described, before outlining the validation strategy and performance evaluation methodology, assessing correctness and comparing the performance of the asynchronous and synchronous communication models.

The report then discusses related work, situating the contribution within existing research, and concludes with a summary of findings, limitations and directions for future work.

Background

2.1 Operating System Kernels

2.1.1 Definition and Characteristics

The kernel is the core component of an operating system, executing at the highest privilege level (if not running on a hypervisor[**?**]) of the processor (typically referred to as supervisor mode[**?**]). It provides the fundamental interface between hardware and software, managing critical system resources, as shown in Figure 2.1.

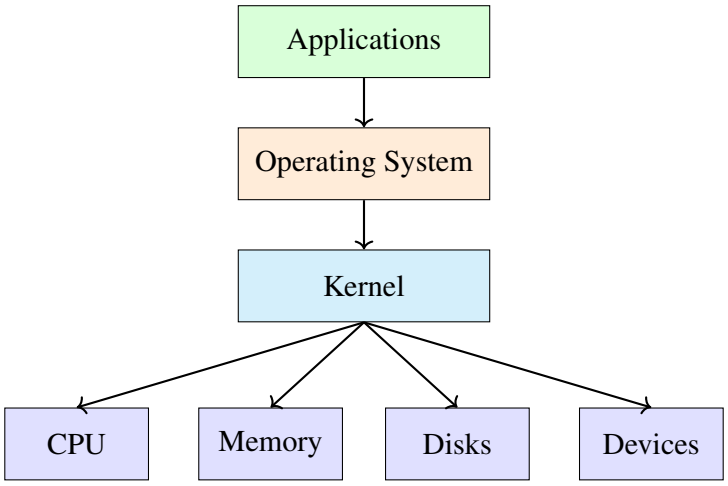

Figure 2.1: View of kernel interaction within an operating system.

The specific set of managed resources varies by system design, but at a minimum the kernel must provide services that cannot be implemented safely or reliably in user space, such as memory management and process or thread scheduling[**?**]. By acting as the sole authority over these resources, the kernel enforces isolation and protection, preventing unintended or malicious interference between processes or to the system as a whole[**?**].

2.1.2 Microkernels Versus Monolithic Kernels

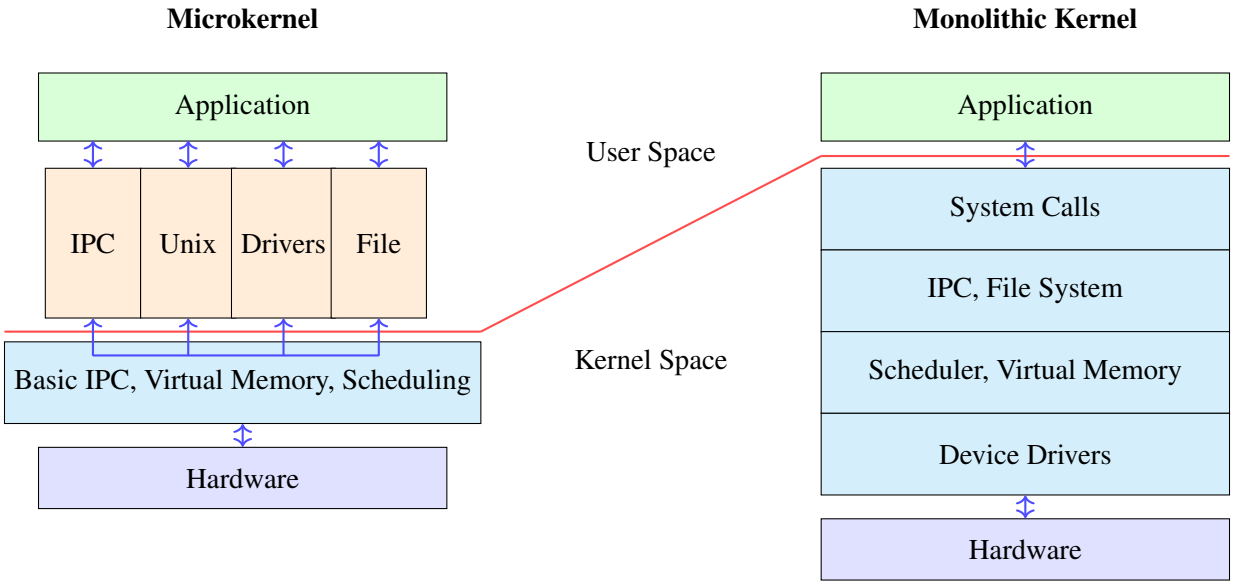

Figure 2.2: Comparison of microkernel and monolithic kernel architectures.

In addition to the required functionality, monolithic kernels also incorporate other low-level services such as device drivers and file systems directly within the kernel address space, as shown in Figure 2.2. All operating system services can then be accessed via system calls[**?**], which transfer control from user space to kernel space through a controlled privilege level switch[**?**]. Because these services execute within the same privileged address space, interactions between kernel components can be performed through efficient function calls, utilising direct access to process data structures. This results in low-overhead service invocation, enabling high-performance systems[**?**].

From a design perspective, monolithic kernels therefore benefit from simpler service integration - yet, this leads to increased complexity as the system grows. The inherent tight coupling of components can make them more difficult to extend, debug, and maintain[**?**], as faults may propagate across sub-system boundaries and be harder to isolate[**?**]. Combined with the larger size of monolithic kernels, this increases the likelihood of defects within the trusted computing base[**?**]. This is significant, as faults within the kernel can have system-wide consequences due to its privileged execution context[**?**].

In contrast, microkernels predominantly restrict kernel functionality to essential mechanisms, delegating higher-level services to user-space components, which must communicate via message passing across their protection boundaries[**?**], as illustrated in Figure 2.2. As a result, requesting a service introduces considerably more overhead when compared to monolithic architectures[**?**]. This primarily arises from context switching and scheduler involvement, when the server resides in a separate process from the client, potentially disrupting processor caches and pipelines[**?**]. However, various optimisations – discussed in Section 2.2 – can significantly reduce these costs.

Additionally, because processes reside in separate address spaces, data may need to be copied or explicitly

shared between them, introducing further overhead. In some cases, this cost can be mitigated through alternative communication mechanisms. Shared memory[**?**], for example, allows processes to exchange data without copying by mapping a common region into multiple address spaces. When combined with techniques such as polling or lock-free data structures[**?**], communication can occur with minimal kernel involvement. These approaches, however, require careful design to ensure correct synchronisation, memory ordering and preservation of isolation guarantees[**?**].

Despite these challenges, the reduced kernel size and strong isolation between components in microkernel architectures enable significantly improved security assurance and fault containment[**?**]. These advantages motivated the design of the L4 family of microkernels, which demonstrated that IPC overhead can be significantly reduced through careful kernel design and optimisation[**?**].

2.2 seL4 Microkernel

2.2.1 Overview and Characteristics

The L3, and subsequently L4, microkernels were developed in response to the poor performance of earlier microkernel designs[**?**], based on the premise that prioritising efficiency was essential for practical adoption. By recognising IPC as the dominant factor in microkernel performance and optimising these mechanisms accordingly, the L4 design achieved approximately a 20× performance improvement over Mach[**?**], the most widely used microkernel at the time[**?**]. This demonstrated that microkernels could achieve competitive performance, leading to significant further research and the development of multiple L4-derived systems[**?**], including seL4.

seL4 (Security-Enhanced L4) extends this architecture by combining the high-performance IPC with comprehensive formal verification[**?**], becoming the only microkernel to do so[**?**]. Through the complete kernel implementation being proven correct against its specification, seL4 ensures that system calls behave as specified and that key safety properties – such as memory isolation – are upheld. Assuming correct hardware behaviour[**?**], this provides a high degree of assurance that kernel integrity and availability are maintained and that information is not leaked between processes, only explicitly shared[**?**].

However, these guarantees apply only to the kernel itself. User-space components are rarely formally verified[**?**] and may still exhibit incorrect behaviour and cause failures at that level. While such faults are isolated and cannot compromise kernel correctness, they may still impact overall system functionality.

This separation of concerns makes seL4 particularly well-suited to exploring user-space approaches to performance optimisation, as the minimal and stable kernel design enables flexibility in higher-level system components, without modifying the verified core.

2.2.2 Design

The L4 specification optimises IPC through furthered kernel shrinkage and efficient message passing[**?**]. Earlier microkernel designs attempted to support richer communication abstractions within the kernel, introducing additional and often unnecessary overhead[**?**]. In contrast, L4 adopts a minimal design, providing only basic, synchronous IPC primitives[**?**]. This approach prioritises low latency and in the common case, removing more complex mechanisms – such as asynchronous buffering and batching – from the kernel[**?**]. These may instead be constructed externally when required, utilising the efficiency of the primitives provided without burdening the kernel.

Kernel involvement is also minimised, with direct process-to-process communication enabled where possible[**?**]. A key aim developed during this design was to ensure that all critical IPC paths fit within the L1 instruction cache[**?**], thereby reducing cache misses and execution latency. Furthermore, Remote Procedure Call (RPC) semantics[**?**] are supported, combining message passing and control transfer into a single operation. This reduces scheduling overhead by enabling direct handoff between client and server threads when a service is invoked.

To make full-system verification realistically possible, seL4 is minimised near completely, more so than even the original L4 design[**?**]. To enforce the security properties specified in its proof, seL4 employs a capability-based access control model[**?**] as shown in Figure 2.3, which associate permission ownership with clients, rather than with objects. Capabilities are unforgeable tokens initially provided by the kernel, where possession of them grants explicit rights to their corresponding kernel objects, instead of a server having a list of client permissions, and clients having credentials to validate their requests with. This removal of ambient user authority eliminates 'confused deputy[**?**]' type problems.

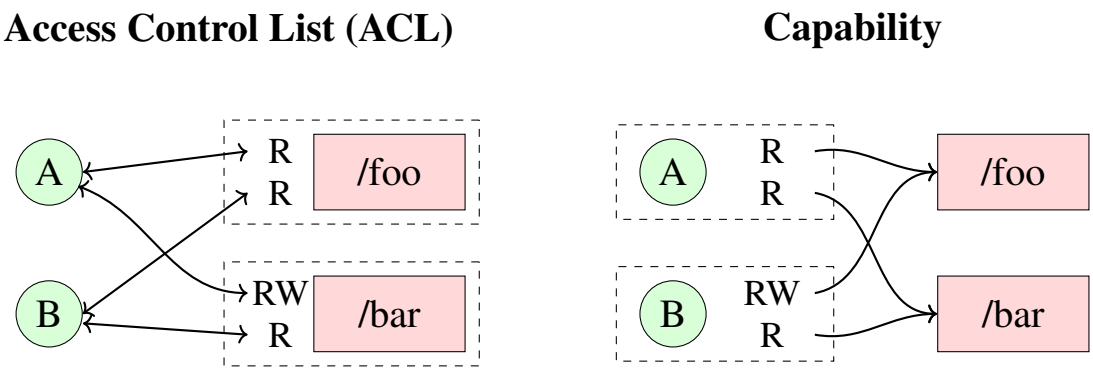

Figure 2.3: Comparison of ACLs and Capability Based Access Control Models.

This distribution of authority also decentralises access control, removing the single-point of failure risk, and simplifies security management by eliminating identity permission checks[**?**]. Following the principle of least privilege[**?**], capabilities are granted by the kernel only when required for operation and may only be transferred explicitly between processes via a request to the kernel, ensuring that authority delegation is controlled and auditable.

2.2.3 Use Cases and Applications

seL4's principles of minimality and high assurance lend themselves particularly well to safety-critical and real-time systems, such as aerospace, automotive, and medical devices, where correctness and isolation are crucial. However, it is also intended as a general-purpose microkernel[**?**] to enable high-performance systems, establishing that strong security guarantees are not a barrier to efficiency.

2.2.4 Microkit Framework for seL4

The seL4 Microkit is a lightweight operating system framework built on top of seL4, designed to simplify and accelerate system development, while still leveraging the microkernel's performance and security benefits. It achieves this by abstracting away many complex, low-level seL4 mechanisms – such as capability management and system initialisation – and manages them automatically under a structured component model[**?**]. As such, systems are composed of Protection Domains (PDs)[**?**], each representing an isolated execution environment containing a thread, an address space and a capability space (CSpace).

To aid this abstraction simplicity, Microkit systems are defined statically through a system description[**?**], from which initialisation code is generated, as displayed in Figure 2.4.

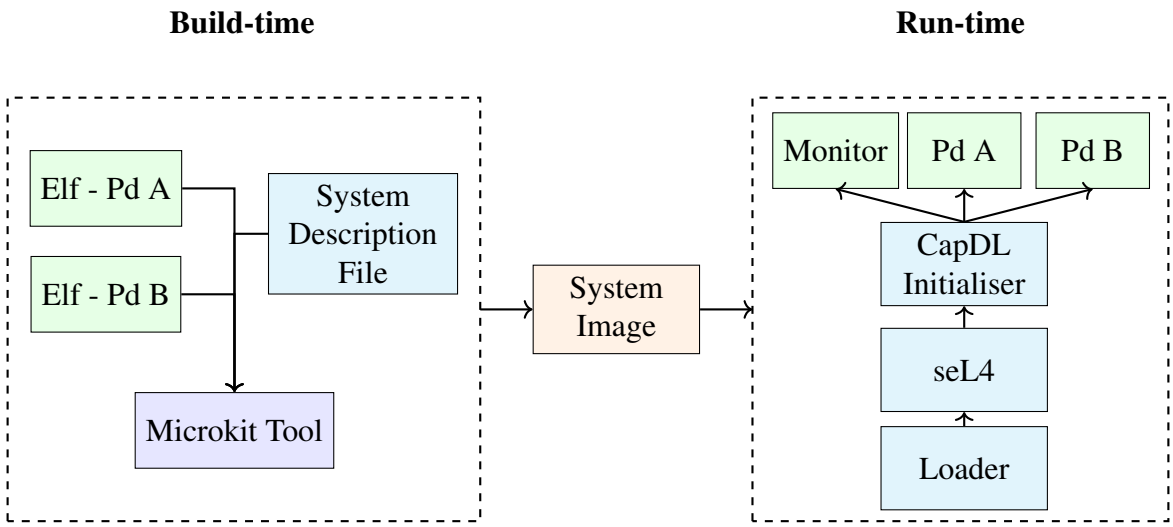

Figure 2.4: Build-time and Run-time Structures for a Microkit-based System.

This enables explicit specification of communication channels and shared memory regions between PDs, easing system construction and improving predictability.

These communication channels between PDs are connected via well-defined entry points[?], as shown in Figure 2.5.

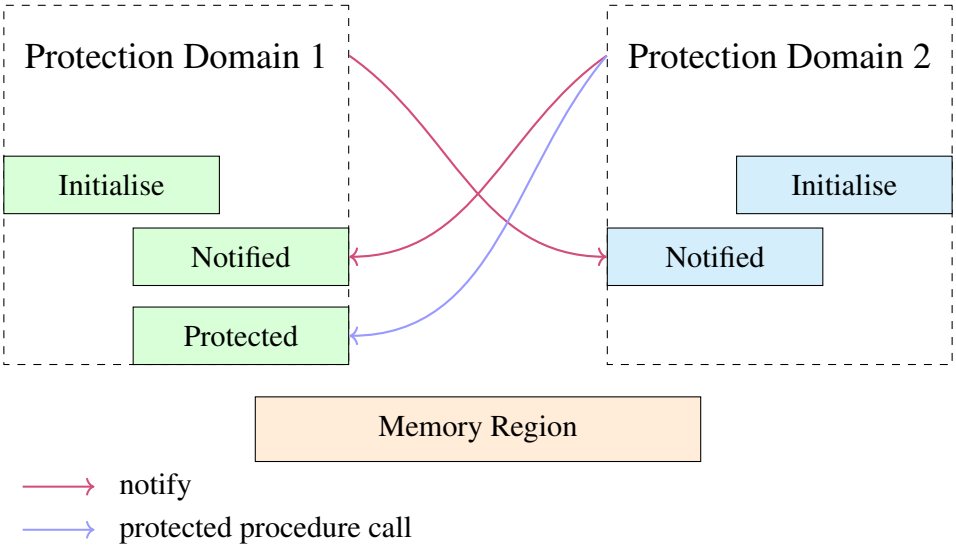

Figure 2.5: Communication Channels Between PDs.

Synchronous interactions are implemented using RPC communication, while asynchronous interactions are supported via notification objects[?], enabling event-driven signalling between components.

2.3 Client-Server Communication

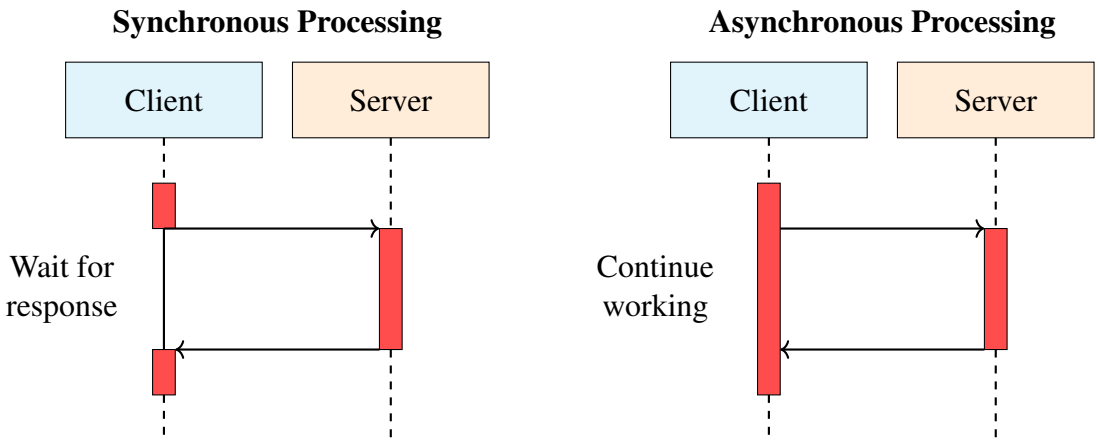

Figure 2.6: Comparison of Synchronous and Asynchronous Communication Models.

2.3.1 Synchronous Model

Synchronous interfacing is a communication model in which the client blocks upon issuing a request and resumes execution only once a response is received from the server, as illustrated in Figure 2.6. This is most

effective in systems where the server is highly available and response latency is low, such that blocking delays are minimised. It is therefore commonly employed in tightly-coupled environments, as they are built around assumptions of strong coordination and shared resources[**?**]. This enforced ordering of operations also significantly improves the predictability of interactions and reduces the system complexity[**?**], hence its use within the seL4 kernel.

It is also ideal for situations where subsequent computation depends on the result of the request. However, when this dependency is absent, performance can be limited by this blocking mechanism due to client idling and high communication overhead for high-frequency operations, as only one may be completed per request.

2.3.2 Asynchronous Model

Alternatively, asynchronous interfacing decouples the client's request from server-side processing, allowing execution to continue independently of response handling, as shown in Figure 2.6, enabling significantly improved system throughput and resource utilisation potential. This effect is amplified in multiclient environments, where multiple clients may issue requests concurrently, avoiding the widespread blocking that arises when the server must service each request synchronously.

In addition, asynchronous designs enable batching, where multiple requests are combined and processed together. By reducing the number of IPC interactions required, batching can significantly lower communication overhead and improve overall efficiency[**?**].

This model aligns naturally with microkernel design, where IPC is a dominant factor in performance[**?**]. As system components are structured as loosely-coupled user-space services, minimising IPC-induced synchronisation delays between services is key to improving overall system efficiency.

However, these benefits come at the cost of increased design complexity. Supporting concurrent and non-deterministic usage patterns requires the use of queues and buffers to manage outstanding requests and responses. These structures must be carefully sized and managed (by both the client and the server) to prevent request loss or backpressure. If insufficient buffering/queueing is available, the system may degrade towards synchronous behaviour, as clients are forced to wait for available capacity.

More generally, asynchronous communication shifts the overhead from client idle time to memory usage: latency due to blocking is reduced, but additional data structures and coordination are required to manage multiple requests.

2.4 File Servers in Operating Systems

2.4.1 Role and Functionality

A file server is a centralised operating system component responsible for managing persistent data storage and providing controlled access to that data for multiple clients, as described in Figure 2.7. Persistence is typically achieved through disk-backed storage, allowing data to outlive the execution of individual processes and, in many cases, system restarts. This enables clients to externalise state once it becomes relatively stable, reducing their memory usage while ensuring durability.

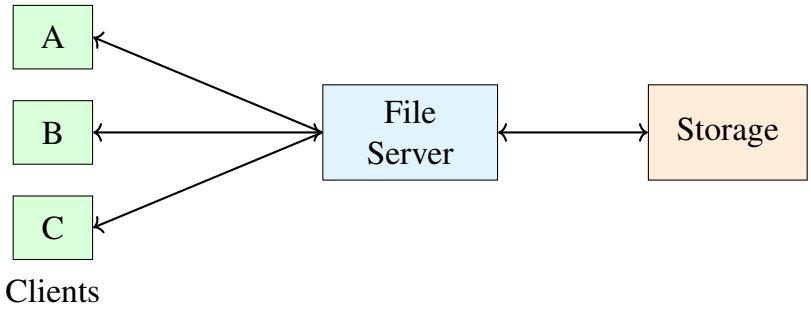

Figure 2.7: Multiclient File Server Structure.

By maintaining data within a shared, centralised repository, the file server facilitates implicit data sharing between clients without requiring explicit inter-process data transfer. This improves efficiency by reducing duplication and minimising communication overhead. However, such a shared access model requires robust mechanisms for access control and isolation. The file server must enforce policies that govern which clients can read, write, or modify specific files, ensuring security, integrity, and consistency in a concurrent multiclient environment.

2.4.2 Abstractions

To facilitate efficient data storage and retrieval, a file system introduces multiple layers of abstraction over raw storage. The primary abstraction is the file, which represents a named, persistent sequence of bytes that is independent of its physical organisation on the underlying medium. This logical abstraction enables interactions with the data as though it were stored contiguously, regardless of fragmentation or layout optimisations performed by the system.

In addition to the byte sequence itself, each file is associated with metadata that describes its properties. This metadata commonly includes attributes such as file size, access permissions and ownership[**?**]. Such information is essential for enforcing access control and supporting higher-level system functionality.

Client interactions with files are mediated through a well-defined set of parameterisable operations, typically including create, open, read, write, close and delete as a minimal interface[**?**]. These operations provide a

consistent and reliable mechanism for managing the file lifecycle and accessing file contents.

To decouple user processes from file server internals[**?**], these operations reference files indirectly via a process-local handle, obtained through an open or create operation. These handles, commonly referred to as file descriptors, additionally encapsulate per-process state associated with an open file instance. This state includes attributes such as the access mode (e.g. read or write) and the current file offset, reducing the need to repeatedly specify this information across operations.

Furthermore, to support client management of multiple files, files are often organised hierarchically within directory structures, providing a namespace that maps human-readable names to underlying file objects. This organisation also allows easier metadata management and lookup operations for groups of files.

2.4.3 Storage and Organisation

File Layout

Unlike the logical view of a file as a contiguous byte sequence, physical storage does not generally store files contiguously. Doing so would make storage management inefficient, as operations such as file growth could require relocating large portions of data if the next bytes are not free. As such, most file systems instead employ a block-based storage model[**?**].

In this model, storage is divided into fixed-size blocks, which can be allocated to files independently. A file is therefore represented as a collection of these blocks, with metadata structures maintaining the mapping between logical byte offsets and their corresponding physical locations. Block sizes (typically around 4KB) are often chosen to align with virtual memory page sizes, allowing more efficient interaction between the file system and memory accesses[**?**].

While this approach improves flexibility and space utilisation, it introduces additional overhead. Sequential data may be physically non-contiguous, potentially reducing read performance due to increased seek or access costs[**?**]. Furthermore, as all accesses occur at block granularity, the choice of block size represents a trade-off between space efficiency and operational overhead. Smaller blocks reduce internal fragmentation and wasted space when storing small files, but increase metadata overhead and the number of accesses required for large files.

Allocation Strategies

There are various methods through which these blocks are mapped to hold the contiguous file data.

Contiguous Allocation

The simplest strategy is contiguous allocation, where a file is stored in a sequence of adjacent blocks. This provides efficient sequential access and hence requires minimal metadata. An example implementation of this is shown in Figure 2.8.

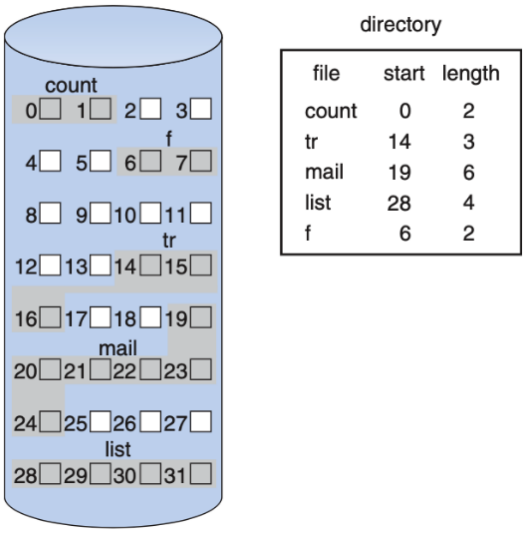

Figure 2.8: Contiguous Allocation Example.

Compared to a contiguous, non-block-based approach, it retains the same requirements for free space, and therefore suffers from similar limitations in file growth and susceptibility to external fragmentation. However, it improves upon this model by introducing fixed-size allocation units, allowing storage to be managed more flexibly at the block level.

Linked Allocation

Linked allocation avoids the limitations of contiguous allocation while maintaining relatively low metadata overhead per file. In this approach, each block contains a pointer to the next, as shown in Figure 2.9, allowing files to grow dynamically as any free block can be allocated.

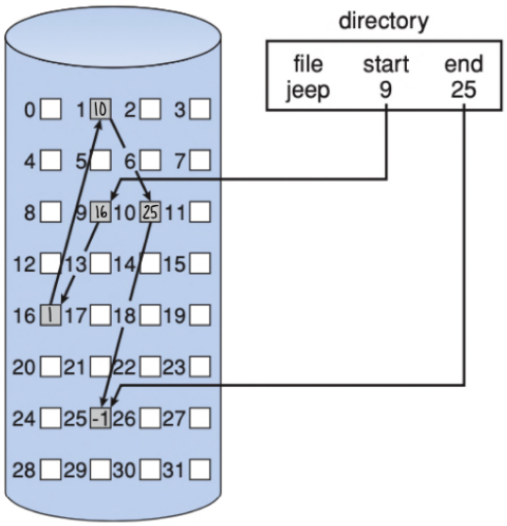

Figure 2.9: Linked Allocation Example.

However, this structure makes random access inefficient, as locating data at a given offset requires traversing the chain of blocks sequentially.

Indexed Allocation

Indexed allocation, which is the most commonly used approach[**?**], addresses the limitations of both contiguous and linked allocation. In this method, each file is associated with a metadata structure that stores references to its data blocks, allowing direct access to any portion of the file, as illustrated in Figure 2.10.

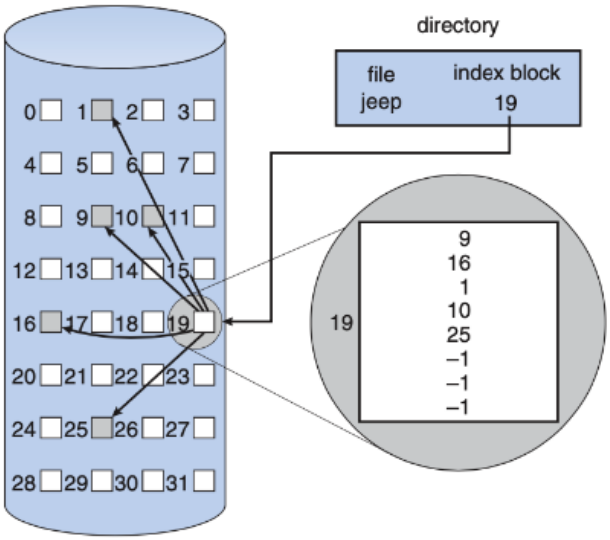

Figure 2.10: Indexed Allocation Example.

This enables both flexible file growth and efficient random access. However, this comes at the cost of increased metadata overhead, as additional storage and lookups are required to resolve block locations.

Metadata Organisation

Metadata may be organised either centrally or on a per-file basis. In centralised approaches, such as File Allocation Table (FAT) systems[**?**], a single global structure stores the mappings for all files. This is achieved using a linked allocation strategy with external pointers, where each entry in the FAT table corresponds to a block and contains a reference to the next block in the file, or an end-of-file marker. An example of this is shown in Figure 2.11.

This means per-file block metadata need only store the index of the file's starting block. While this approach is simple and centralises metadata management, it inherits the limitations of linked allocation.

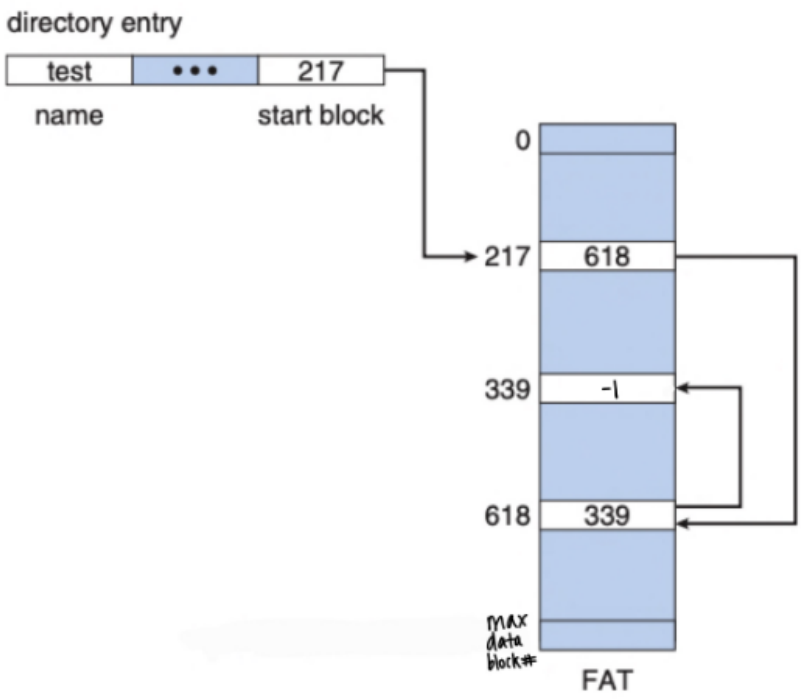

Figure 2.11: File Allocation Table Example.

Alternatively, per-file metadata strategies, such as inode-based systems[**?**], associate each file with a dedicated metadata structure that stores both file attributes and references to its data blocks. These references may be stored using a combination of direct and indirect pointers. Direct pointers reference data blocks directly, but as inodes are fixed-size structures, their number is limited. To support larger files, indirect pointers are used, where a pointer instead references a block containing further block addresses. This is illustrated in Figure 2.12.

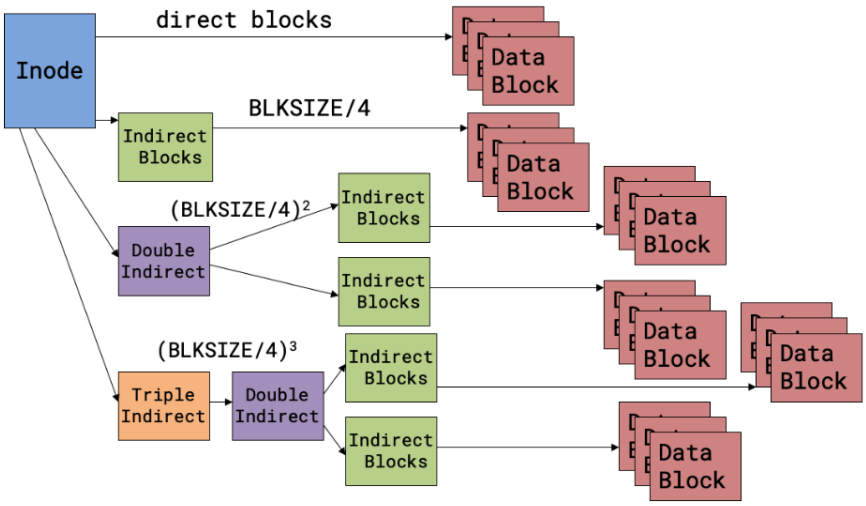

Figure 2.12: Indirect Block Indexing Example.

While this enables scalable file representation, it introduces additional levels of indirection, increasing the overhead required to resolve data locations.

Hybrid approaches of these are also common such as New Technology File System (NTFS)[**?**], which superseded FAT in Windows. NTFS employs per-file metadata structures similar to inodes, but represents file data using extents, storing ranges of contiguous blocks rather than listing each block individually, reducing metadata overhead for large, contiguous files.

Directory Structure

A directory is a structure that stores references to files rather than file data, enabling files to be organised hierarchically through the use of subdirectories, as demonstrated in Figure 2.13.

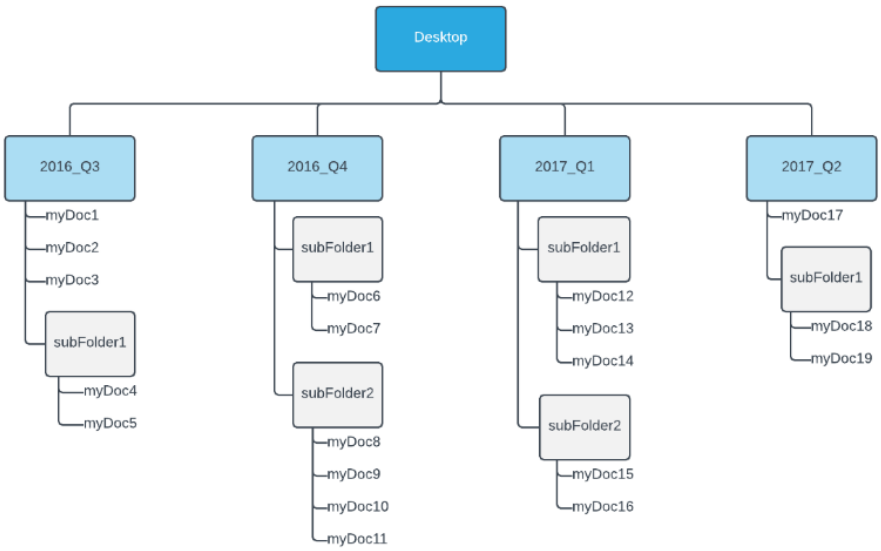

Figure 2.13: Directory Structure Example.

This is achieved via directory entries, which associate a human-readable name with either a subdirectory or a file's corresponding metadata structure, either directly in the entry for centralised systems or via a reference in per-file systems. Path-based access is used to locate files, where a path represents a sequence of directories that must be traversed to resolve a name to its associated metadata.

2.4.4 Client Management

As previously discussed in Section 2.4.2, metadata stores information such as file ownership and access permissions. These are required to ensure that clients can control access to their files and prevent unauthorised modification or retrieval by others. At the simplest level, files may be designated as private, where only the owner has access, or public, where any client may access them.

More fine-grained systems define separate permissions for operations such as reading, writing and execution, allowing, for example, a file to be read but not modified by other clients. More advanced models, such as capability-based access control[**?**] or access control lists (ACLs)[**?**], allow permissions to be specified for individual clients or groups, providing greater flexibility at the cost of increased complexity in storage and management.

Additionally, when multiple clients have write access to a file, mechanisms may be employed to prevent race conditions[**?**] and ensure consistency, particularly for deletion operations.

2.5 Related Work

Existing file systems and I/O services are commonly implemented using synchronous request-response models, where operations block until completion[**?**]. While this simplifies reasoning and integration with existing software, it can limit concurrency and lead to inefficient resource utilisation. Recent developments such as `io_uring` have demonstrated significant performance improvements by enabling asynchronous I/O through submission and completion queues, reducing syscall overhead and allowing multiple outstanding operations[**?**]. However, this shift in execution model has introduced security concerns, as existing security mechanisms were not designed for such interfaces, leading to vulnerabilities arising from increased complexity and a larger attack surface[**?**].

In contrast, seL4 is a comparatively recent system designed with strong security guarantees and minimal trusted computing base. As a result, it lacks much of the legacy infrastructure that constrains the adoption of new interfaces in more established systems[**?**]. Furthermore, unlike some other microkernel designs, seL4 does not provide asynchronous primitives at the kernel level, instead relying on synchronous IPC. While this design performs well for individual interactions, it can limit concurrency in user-space services where operations may require multiple sequential IPC calls, and have multiple clients.

Consequently, introducing asynchronous behaviour at the user level presents a promising approach to improving seL4 performance without compromising the kernel's design principles. However, there is limited existing work exploring asynchronous, multiclient file server designs in seL4 specifically targeting the reduction of IPC overhead.

Design

3.1 System Requirements and Constraints

The design of the file server is driven by both functional requirements and the constraints imposed by the Microkit framework for the seL4. These requirements define the expected behaviour of the system, while the constraints shape the design decisions necessary to achieve this within the given environment.

From a functional perspective, the system is required to provide a core set of file system operations, including creation, reading, writing and deletion of files, alongside support for directory structures and getting and setting associated metadata such as permissions and file size. The system must present a consistent interface to clients, ensuring that operations are executed correctly and that appropriate error codes are returned in response to invalid inputs or failure conditions.

A key requirement of the system is support for multiple clients operating concurrently. The file server must therefore be capable of handling requests from multiple independent clients, maintaining correctness and consistency in the presence of interleaved operations. In addition, the system must support both synchronous and asynchronous interaction models. This enables direct comparison between traditional blocking request–response behaviour and the proposed asynchronous, batched approach.

Performance is a central non-functional requirement. In particular, the system is designed to minimise the overhead associated with inter-process communication (IPC), which is a known bottleneck in microkernel-based systems. The asynchronous design must therefore enable efficient batching of operations, reducing the number of required IPC interactions and improving overall throughput. The system must also scale to multiple clients without significant degradation in performance.

Several constraints significantly influence the system design. The Microkit framework enforces a statically defined system configuration, meaning that all protection domains, communication channels and memory regions must be specified at build time. As a result, the number of clients and their associated resources are fixed.

The IPC model provided by seL4 is inherently synchronous, with communication occurring through blocking interactions. This necessitates the use of shared memory and notification mechanisms to enable non-blocking request submission and completion handling.

Further constraints arise from the execution environment. For development simplicity, the system operates with no access to persistent storage, resulting in a purely in-memory file system. Additionally, due to seL4 constraints, execution is limited to a single core, restricting the ability to exploit parallelism. This requires the system to rely on batching and efficient communication patterns, rather than parallel execution, to achieve performance improvements.

These requirements and constraints directly inform the design of the system, motivating the use of asynchronous batching, shared memory communication and a modular file system structure to balance performance, correctness and simplicity within the limitations of the Microkit environment.

3.2 IPC Design and File Server Interface

3.2.1 Synchronous Model *(Baseline)*

3.2.2 Asynchronous Model *(Proposed IPC Mitigation Strategy)*

3.3 File Server

3.3.1 Architectural Overview

3.3.2 Component Design and Interactions

3.3.3 Multiclient Support

3.3.4 File and Directory Operations

Implementation

4.1 Development Environment and Tools

The system was implemented in C using the Microkit framework for seL4, specifically version 2.0.1 of the Microkit SDK[**?**]. Development targeted the AArch64 architecture[**?**], with code compiled using the aarch64-linux-gnu cross-compilation toolchain. The build process was managed using a Make-based system, which coordinated compilation, linking and generation of Microkit system images.

All development and testing were conducted in an emulated environment using QEMU[**?**], targeting the virt platform with a Cortex-A53 CPU configuration. This allowed rapid iteration without requiring physical hardware, though it limits evaluation of hardware-specific performance characteristics.

Debugging and verification were primarily performed using logging via serial output, enabling inspection of system behaviour during execution. Additional support tools included Git[**?**] for version control and reproducibility of development progress.

4.2 Microkit Framework Setup

The system is built using the Microkit framework for the seL4, which provides a declarative mechanism for defining system structure, memory layout and inter-process communication[**?**]. A Microkit system is specified using an XML-based description, which is used to generate a complete system image comprising multiple protection domains.

Each protection domain represents an independent user-space process with its own address space and execution context. In this system, separate protection domains are defined for the file server and each client. Program images are statically assigned to each domain, and memory regions are explicitly declared and mapped into their address spaces at specified addresses. A dedicated memory region for file system state is declared and mapped solely to the server. Additionally, for each client, regions to hold submission and completion queues and associated buffers are declared and mapped into both the file server and the corresponding client, each with the appropriate permissions, allowing shared access.

Alongside this shared memory, communication between protection domains is established using channels, which define endpoints for inter-process communication, with each client being connected to the file server. For the synchronous system, protected procedure calls[**?**] are used to perform synchronous message passing

to request file operations and block until a response is received. In contrast, the asynchronous design utilises notification objects[**?**], which provide non-blocking signalling to indicate when requests have been submitted or responses are available.

As Microkit systems are statically defined, the number of clients and their associated resources must be specified at build time. To support varying workloads, a script was developed to automatically generate the system description for a configurable number of clients, ensuring consistency while avoiding manual duplication.

In addition to structural configuration, Microkit also requires specification of priority and scheduling parameters for each protection domain. These priorities determine the order in which domains are scheduled by the seL4 kernel, directly influencing system responsiveness and fairness. In this system, the file server is assigned a higher priority than client domains to ensure that incoming requests are serviced promptly, reducing queuing latency and preventing clients from stalling due to delayed processing. Client domains are assigned equal priorities relative to one another to maintain fairness across request sources. This scheduling configuration is particularly important in the asynchronous design, where the server must efficiently respond to notifications and process queued operations without being preempted excessively. Consequently, as seL4 utilises a round-robin style scheduling policy[**?**], careful time budget allocation ensures that IPC handling remains responsive while still allowing clients sufficient execution time to generate and consume requests.

4.3 Interface

4.3.1 Synchronous Server Model

Client-Server Communication Protocol

IPC Mechanism

API

4.3.2 Asynchronous Server Model

Client-Server Communication Protocol

IPC Mechanism

API

4.4 File Server

4.4.1 Data Structures and Access Methods

inodes

lorem

Blocks

Directories

File Descriptors

4.4.2 Operations

Entry Creation

File Access

Metadata Access and Modification

Entry Deletion

4.5 Error Handling

The primary sources of errors in this system arise from invalid client requests and resource limitations. These include cases such as invalid permissions, malformed paths or file descriptors, as well as exhaustion of resources such as available blocks or file descriptor space. To maintain correctness, these conditions are explicitly checked before and during the execution of each operation.

To support this, operations are decomposed into smaller functions, each responsible for a specific task. At each potential point of failure, inputs and system state are validated, with failures resulting in the return of a defined error code from a global set. These error codes are propagated through the call chain, allowing higher-level operations to respond appropriately. Where possible, alternative actions are attempted; otherwise, the operation is safely terminated, reverting any partial changes to maintain consistency. The final error state is then communicated back to the client through the response structure. Some errors such as trying to read a size larger than the buffer size do not terminate the operation, but the client is still notified that it was not fully successful.

Evaluation

5.1 Validation

Extensive validation was conducted to ensure the correctness, robustness and reliability of the file server under both normal and adverse conditions. Given the range of supported operations, numerous edge cases and error scenarios arise. Additionally, the presence of multiple clients and asynchronous communication introduces further complexity, requiring testing not only to verify functional correctness but also to ensure that the system maintains consistency and stability under concurrent workloads.

To increase confidence in the correctness of the system as a whole, testing was conducted at multiple levels. Initially, unit testing was used to verify the functionality of individual components in isolation. This enabled a modular development approach, allowing components to be validated incrementally during implementation. Testing at this level also simplified debugging, as errors were easier to localise due to their limited scope and the reduced code surface.

However, as the system relies heavily on interactions between components and communication between protection domains, integration testing was also necessary. This enabled validation of end-to-end behaviour, ensuring that all operations functioned correctly when executed through the full client–server interface under a range of conditions.

5.1.1 Unit Tests

Unit tests were developed for the core of the file system, including components such as block management, inode management, directory handling and file table operations. These tests focused solely on verifying the correctness of the file server's data structures and their accesses.

To ensure this isolation, these components were tested by directly invoking their functions outside of the Microkit environment. This allowed inputs and supporting data to be mocked through deterministic flows, and the internal state of the file system to be inspected, enabling correctness to be validated at each stage of execution. Testing regular inputs alongside failure conditions such as resource exhaustion gave confidence that the fundamental building blocks of the file system operated correctly before integration and would not cause wider issues upon failure.

For example, one test constructs a mock “indirect block” containing a list of allocated block indices, invokes the indirect-release routine, and then verifies that each referenced block is marked free in the allocation table.

5.1.2 Integration Tests

Once the components were verified, integration testing was used to validate the behaviour of the system as a whole, exercising all file system operations through its regular running microkit environment. Rather than directly inspecting internal state, validation was performed through the responses and error codes returned to clients, such as creating a file then listing the file system. This ensured that testing reflected real system usage and confirmed that correct behaviour was exposed through the public interface.

Single Client

These tests exercised the majority of functionality from the client’s perspective, covering the full lifecycle of files, including creation, reading, writing and deletion, without the added complexity of concurrent access. Within this scope, it was possible to verify that invalid client inputs were correctly rejected with appropriate error codes, for example when attempting to delete a non-existent file. Additionally, boundary conditions were tested to ensure correct behaviour under extreme cases, such as reading the maximum supported amount of data.

Multiple Clients

To ensure that all clients were serviced correctly and fairly, and that the system operated reliably under load, extended tests were conducted which interleaved operations from multiple clients. This enabled validation of scenarios that cannot be observed in a single-client context, including permission enforcement and modification of shared resources.

As clients execute independently, explicit synchronisation was required. This was achieved through the use of marker files, where clients created files with predefined names that were monitored by other clients. These acted as barriers, allowing coordination of execution order and enabling clients to signal completion of operations to both the server and one another. For example, one test creates a publicly readable file within client 1, after which client 2 attempts to read the file to verify access is permitted. The file is then modified by client 1 to be private, and client 2 subsequently attempts to read it again, verifying that the operation is correctly rejected with a permissions error.

5.2 Performance Evaluation

5.2.1 Goals

The primary goal of this evaluation was to compare the performance characteristics of the synchronous and asynchronous file server designs. In particular, the evaluation aimed to quantify the impact of inter-process communication (IPC) overhead and assess the effectiveness of batching in reducing per-operation cost. Additionally, the behaviour of both systems under multiclient workloads was examined to determine how well each design scales under contention.

5.2.2 Metrics

For the per operation benchmarking, performance was evaluated using the average time per operation, calculated by measuring the total execution time of a long sequence of operations, to amortise any stochastic effects, and dividing by the number of operations performed. For the more realistic workflows, time taken to complete the whole suite was used. These provided consistent bases for comparing synchronous and asynchronous implementations.

Time measurement was performed using the CPU cycle counter, as the low-level execution environment does not provide native timing facilities. The cycle count was recorded before and after each client's measurable operations, and converted to elapsed time using the processor's clock frequency. Results were averaged across multiple runs to reduce variance and improve reliability.

In multiclient scenarios, fairness was also considered. This was validated by recording the total time taken by each client to complete its workload, ensuring that no individual client experienced disproportionate delay.

5.2.3 Experimental Setup

All experiments were conducted within the same emulated environment using QEMU on an AArch64 platform, ensuring consistency across all tests. Both synchronous and asynchronous implementations were evaluated under identical conditions, with the same workloads, file sizes and file system state.

For each experiment, the only variables altered were the server design (synchronous or asynchronous), the batching configuration and the number of clients. All operations were validated for correctness during execution to ensure that performance results reflected valid system behaviour rather than erroneous outcomes. During the varied workload, this processing also mimicked non-file I/O that would occur in a typical system, helping to ensure that the measured performance reflects realistic operating conditions.

5.2.4 Benchmark Design

Two benchmark suites were developed to evaluate system performance. The first was designed to isolate and quantify the reduction in communication overhead achieved by the asynchronous server, while the second evaluated overall system performance under more realistic and varied workloads.

This initial aim was achieved through a single-client benchmark that repeatedly executed a single type of operation allowing controlled measurements of individual operation costs for both systems. Both read and write operations were evaluated independently, as they are representative of common file system usage and exhibit differing behaviour. As timing was recorded from the client's perspective, this included both internal server processing time and communication overhead.

For the asynchronous implementation, the benchmark was executed with varying batch sizes. A batch size of one provided a baseline comparison, ensuring that the asynchronous design does not significantly degrade performance when batching is not possible, which is important as many real workloads contain inherent dependencies. Larger batch sizes allowed evaluation of how batching reduces per-operation cost, identifying the point at which performance improvements become significant and where further increases yield diminishing returns.

As the internal file system operation of both systems are the same, this benchmark also enabled estimation of communication overhead. Let s denote the time taken for a single operation in the synchronous system and a_n the time taken for a batch of n operations in the asynchronous system. Let x represent the internal processing time per operation, and c_s and c_{a_n} the communication overheads. Then:

$$s = c_s + x \tag{5.1}$$

$$a_n = c_{a_n} + nx \tag{5.2}$$

And

$$s \cdot n = c_s \cdot n + nx \tag{5.3}$$

Hence

$$s \cdot n - a_n = c_s \cdot n - c_{a_n} \tag{5.4}$$

Using measured values of s and a_n, this relationship allows the difference in communication overhead between the synchronous and asynchronous systems to be quantified across varying batch sizes.

Furthermore:

$$\frac{a_n}{n} = \frac{c_{a_n}}{n} + x \tag{5.5}$$

This formulation also allows analysis of the average cost per operation in the asynchronous system. As the batch size increases, the communication overhead is amortised across multiple operations, causing the per-operation communication cost to decrease. In the limit, the average operation time approaches the internal processing time x, indicating that communication overhead becomes negligible. This highlights the fundamental advantage of batching, as it allows the system to approach optimal performance bounded only by computation.

The second benchmark was a multiclient, realistic workload designed to simulate typical file system usage. Multiple clients executed independently, each performing a sequence of varied operations across many files. This sequence consisted of CREATE, WRITE, CLOSE, OPEN, SEEK, READ and DELETE operations, representing a complete file lifecycle. Operations were interleaved across clients to introduce contention and better reflect real-world usage patterns.

As batching is central to the performance benefits of the asynchronous design, this benchmark was structured to allow batching where it would naturally occur, without artificially constraining the workload. While it is possible to construct workloads with strict dependencies between operations, this would reduce asynchronous behaviour to that of the synchronous system and provide limited insight beyond the single-operation benchmark. Instead, operations were grouped into realistic batches, for example: CREATE — WRITE and CLOSE — OPEN — SEEK and READ and DELETE.

Performance was measured across varying numbers of clients, enabling analysis of scalability, contention and the impact of concurrent access. This approach provided a balanced evaluation, allowing direct comparison of synchronous and asynchronous designs under both controlled and realistic conditions.

5.2.5 Results and Analysis

Single-Client Per-Operation Benchmark

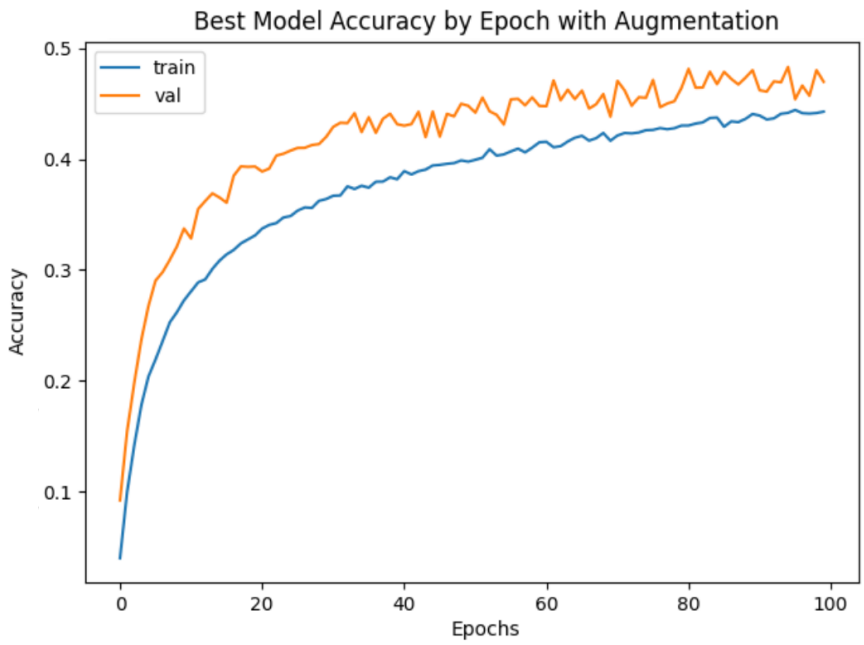

Figure 5.1: Average Time per Operation vs Batch Size (Read and Write)

The synchronous baseline establishes the per-operation cost of IPC without batching. Write operations averaged 186 μs, while read operations averaged 155 μs, reflecting the expected asymmetry due to additional data movement and persistence overhead in writes.

When the asynchronous system is configured with a batch size of one, performance is effectively equivalent to the synchronous implementation. Writes measured 187 μs and reads 158 μs, indicating that the introduction of asynchronous infrastructure does not impose significant additional overhead in the absence of batching. This validates the design goal that the asynchronous system should degrade gracefully to synchronous-like behaviour when batching is not possible.

As batch size increases, a clear and substantial reduction in per-operation cost is observed. For write operations, the average latency decreases from 187 μs (batch = 1) to 74 μs (batch = 10), representing approximately a 60% reduction. Similarly, read operations decrease from 158 μs to 41–48 μs, corresponding to a $\sim$70–74% reduction at optimal batch sizes.

The results exhibit a diminishing returns pattern. The most significant improvements occur between batch sizes 1 and 4:

- Write latency drops from 187 μs to 91 μs ($\sim$51% reduction)

- Read latency drops from 158 μs to 63 μs ($\sim$60% reduction)

Beyond this point, gains become incremental. For example, increasing batch size from 8 to 10 yields negligible improvement for writes (74 μs $\rightarrow$ 74 μs) and even slight regression for reads (41 μs $\rightarrow$ 48 μs), likely due to increased queuing delay or cache effects.

These results align closely with the analytical model:

$$\frac{a_n}{n} = \frac{c_{a_n}}{n} + x$$

As n increases, the amortised communication cost $\frac{c_{a_n}}{n}$ diminishes, and the average operation time approaches the internal processing cost x. Empirically, the asymptotic region appears around batch sizes 7–9, suggesting that beyond this range, IPC overhead becomes a minor component of total latency.

Using the relationship:

$$s \cdot n - a_n = c_s \cdot n - c_{a_n}$$

we can infer that the synchronous system incurs a linear growth in communication overhead with respect to operation count, whereas the asynchronous system effectively flattens this growth. The widening gap between $s \cdot n$ and a_n as n increases directly quantifies the reduction in IPC overhead achieved through batching.

Notably, read operations benefit more significantly from batching than writes. This suggests that read operations are more heavily dominated by IPC overhead in the synchronous system, whereas writes retain a larger proportion of intrinsic processing cost.

Multiclient Realistic Workload

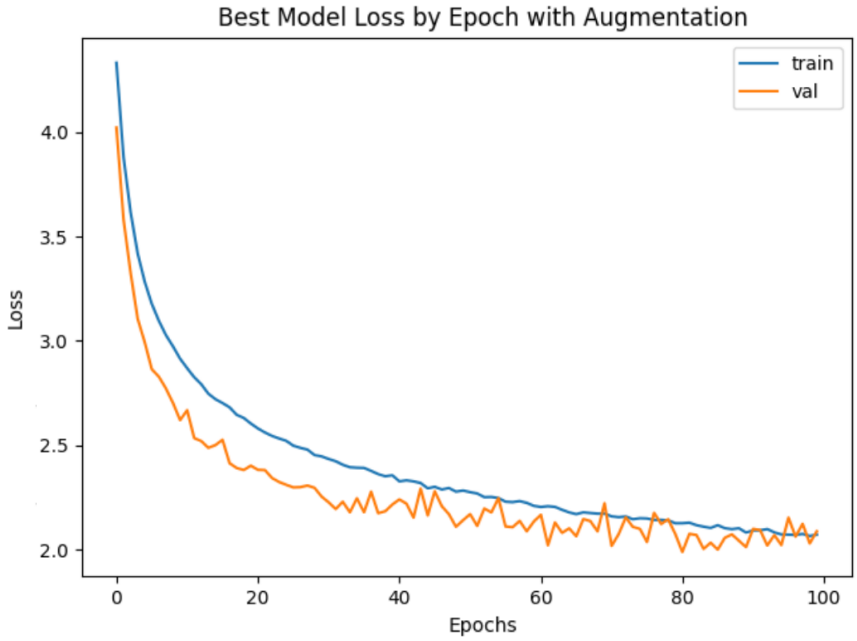

Figure 5.2: Total Execution Time by Number of Clients (Synchronous vs Asynchronous)

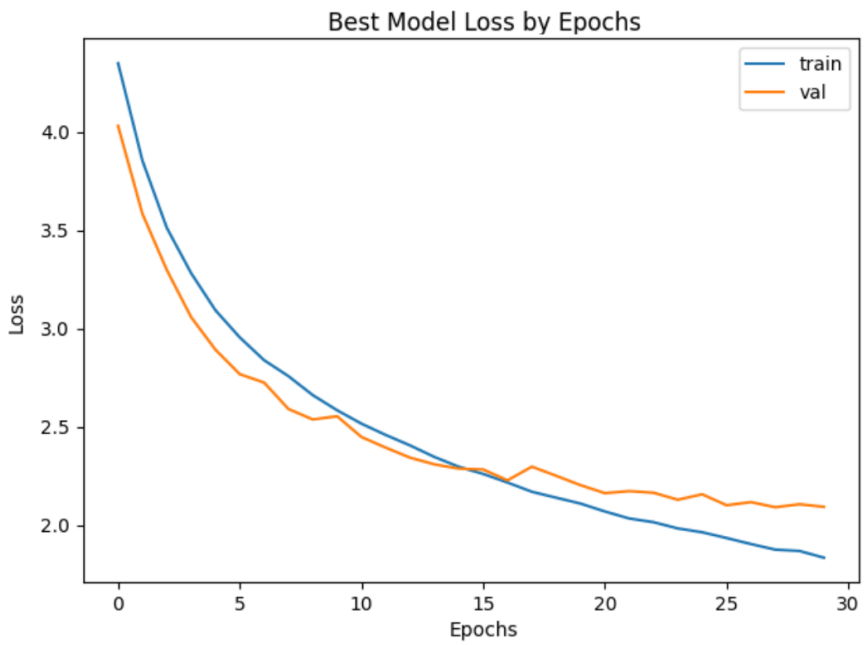

Figure 5.3: Execution Times per Client (Fairness Analysis)

Under multiclient workloads, the asynchronous design demonstrates a substantial performance advantage. The synchronous system exhibits total execution times clustered tightly around 25.3–25.7 million μs, with

per-client averages between 12,680–12,862 μs. In contrast, the asynchronous system completes the same workload in approximately 16.0–16.3 million μs, with per-client averages between 7,844–7,941 μs.

This corresponds to an overall performance improvement of approximately 36–38%, which is consistent with the reductions observed in the single-operation benchmark when moderate batching is achievable.

Fairness across clients is well maintained in both systems. The variation in per-client average time is minimal:

- Synchronous: spread of $\sim$182 μs ($\sim$1.4%)
- Asynchronous: spread of $\sim$97 μs ($\sim$1.2%)

This indicates that neither system introduces significant starvation or prioritisation bias. Importantly, the asynchronous design preserves fairness despite its more complex scheduling and batching behaviour.

Another notable observation is the consistency of asynchronous performance across clients. The tighter clustering suggests that batching and concurrent handling smooth out transient delays, reducing sensitivity to contention patterns. In contrast, the synchronous system, while still fair, shows slightly higher variability, likely due to clients serialising on IPC interactions with the server.

Conclusion

This dissertation set out to investigate whether inter-process communication (IPC) overhead in the seL4 microkernel could be mitigated through architectural and interface-level design choices, specifically via an asynchronous, multiclient file server. In doing so, it involved the design and implementation of a complete system, encompassing not only the file server itself but also the surrounding infrastructure required to support client interaction, memory sharing and communication within the Microkit framework. By reducing the number of synchronous interactions and enabling batching and decoupling of requests, the work demonstrates that meaningful performance improvements are achievable even within the constraints of a formally verified microkernel environment.

The results show that IPC overhead is not merely a function of the underlying kernel primitives, but is significantly influenced by higher-level system design decisions, particularly how clients interact with services, how requests are scheduled and processed and how system components are structured to minimise unnecessary communication. Furthermore, the development of a full, working system highlights that such optimisations must be considered holistically, as performance gains emerge from the interaction between interface design, scheduling behaviour and data management rather than from any single component in isolation.

6.1 Contributions

6.1.1 System Design and Implementation

This work presents a complete user-space file server built using the seL4 Microkit framework, designed with both synchronous and asynchronous interfaces. The system supports multiple concurrent clients, a structured file system abstraction and a reasonably comprehensive set of file operations including file data, directory, lifecycle and metadata management.

Key contributions include:

- A modular file system architecture, that can scale and adapt flexibly through its incorporation of block management, inodes and file descriptors, suitable for microkernel deployment
- A multiclient design supporting permission enforcement and organised client state management
- Dual interface implementations: a traditional synchronous IPC model and an asynchronous model based on notifications, queues and shared buffers
- A flexible API layer enabling transparent, simple use of either communication paradigm
- A full implementation within Microkit, demonstrating practical use of channels, notifications, scheduling and memory management

The implementation validates that complex, stateful services such as file systems can be realised entirely in user space within seL4, while maintaining structured interaction patterns and correctness guarantees.

6.1.2 IPC Overhead Mitigation Insights

The primary analytical contribution lies in demonstrating how asynchronous communication reduces IPC overhead through batching and reduced synchronisation frequency.

The evaluation yields several key insights:

- Baseline parity: Asynchronous communication with batch size 1 performs similarly to synchronous IPC, confirming minimal overhead from the abstraction itself
- Batching effectiveness: Increasing batch sizes significantly reduces per-operation latency, demonstrating amortisation of IPC costs
- Low batch size required for performance maximisation: Performance improvements plateau beyond moderate batch sizes, indicating practical upper bounds for batching efficiency
- Workload sensitivity: The benefits of asynchronous IPC are highly dependent on workload characteristics, especially the degree of operation parallelism and batchability
- Multiclient scalability: In realistic multiclient scenarios, asynchronous design also substantially reduces average operation latency compared to synchronous approaches

These findings reinforce that IPC overhead mitigation is best addressed at the system design level rather than solely through kernel optimisation[**?**].

6.2 Drawbacks and Limitations

Despite achieving its objectives, the system has several limitations that constrain both performance and general applicability.

6.2.1 File System Limitations

The implemented file system is functionally rich and reasonably well validated, but could be improved in multiple areas. Additional operations such as file renaming and copying could be introduced to improve usability. Furthermore, lookup performance is suboptimal due to linear search mechanisms; the absence of more efficient indexing structures (e.g. hash tables or trees) contributes to increased latency as the file set grows, yet the current implementation is sufficient for general usage and the experiments undertaken in this report.

6.2.2 Benchmarking Constraints

Evaluation required the development of custom benchmarks due to the specialised environment, low-level platform and non-standard interface. While these benchmarks effectively demonstrate relative performance trends, they limit direct comparability with existing systems. Consequently, results should be interpreted as indicative rather than absolute.

6.2.3 Lack of Persistent Storage

The system operates entirely in memory, with no backing disk. Incorporating persistent storage would introduce additional complexity, particularly around buffering strategies, write-back policies and cache management. This would likely require a significantly larger cache and careful coordination between storage and IPC layers, representing a substantial extension beyond the current scope.

As such, the absence of disk I/O results in highly deterministic and relatively low operation latencies. In a realistic system with persistent storage, operation timings would be significantly longer and less deterministic due to device latency, queuing effects and variability in access times. As a result, the relative impact of IPC overhead would be reduced, since communication costs would form a smaller proportion of total operation latency. Consequently, while the results clearly demonstrate IPC-related performance trends, they likely represent an upper bound on the benefits of communication overhead reduction in real-world storage-backed systems.

However, such systems would typically operate on multi-core hardware, unlike the single-threaded environment used in this work. In these settings, an asynchronous design such as the one presented here, would considerably offset these reduced benefits by enabling significantly higher throughput through concurrent execution.

6.2.4 Single-Threaded Execution

The server operates on a single thread, restricting its ability to exploit concurrency. This inherently limits throughput and reduces the potential benefits of the asynchronous design. In a multi-core system, multiple clients could concurrently issue requests and process responses in parallel with the server executing operations, significantly increasing overall system throughput. In contrast, the synchronous model effectively serialises access, as clients must block and wait while the server processes each request, limiting concurrency at the system level.

The architecture presented here is, however, designed to be transferable to a multicore environment with only minor modifications, once such support is more readily available within seL4.

This limitation also impacts the interpretation of benchmark results. The observed performance improvements arise primarily from reduced communication overhead through asynchronous batching, rather than from true parallel execution. As a result, the measured gains likely represent a conservative estimate of the performance improvements achievable in a fully concurrent system, where additional benefits from overlapping request generation, processing and response handling would be realised.

6.2.5 Dependency on Workload Characteristics

The effectiveness of the asynchronous model is strongly workload-dependent. In scenarios with low concurrency or minimal opportunity for batching, the advantages over synchronous IPC diminish. This indicates that asynchronous designs are not always superior, but instead most effective in systems with naturally parallel or high-throughput workloads.

6.2.6 Additional Client Burden

The asynchronous design introduces additional complexity on the client side, primarily due to the need to manage request queues and response buffers. Each client must maintain its own data structures to support batching, resulting in an increase in per-client memory usage compared to the synchronous model, where requests are issued and resolved immediately hence requiring only a single buffer and no operation queue.

Although a supporting library was implemented to abstract away much of this functionality, the responsibility for correct and efficient usage still lies with the client. In particular, clients must determine how to group operations into batches, which is inherently workload-dependent and not always straightforward.

Additionally, clients must decide when to flush queued requests and when to synchronise to collect results. Delaying synchronisation increases batching efficiency but introduces latency in observing results, while frequent synchronisation reduces batching opportunities and reintroduces IPC overhead. This creates a trade-off between throughput and responsiveness that must be carefully managed.

Overall, while the asynchronous model provides clear performance advantages, it shifts part of the system

complexity from the server to the clients. This increases the burden on client implementation and may make the system harder to use correctly, particularly for developers unfamiliar with asynchronous design patterns or the underlying IPC mechanisms.

6.3 Future Work

Future work should focus on extending the system to a multicore environment to fully realise concurrency benefits. Additionally, enhancing the file system with additional operations and more efficient data structures, such as tree-based indexing, would improve scalability, and introducing a caching layer and persistent storage would enable more realistic deployment scenarios. Further exploration of IPC mechanisms, including hybrid synchronous—asynchronous models and adaptive batching strategies, could yield additional performance improvements.

Cite some references [1, 2]

Bibliography

[1] W. Dyke. The nutter's guide to LaTeX. *Read Only*, (7), May 1994.

[2] LaTeX. Team. URLs in BibTeX bibliographies. `http://www.tex.ac.uk/cgi-bin/texfaq2html?label=citeURL`, 2012.

Appendix Title

A.1 Example First Section

TODO:

A.2 Output

TODO: